

Informed search algorithms (Heuristic search)

Unit-3

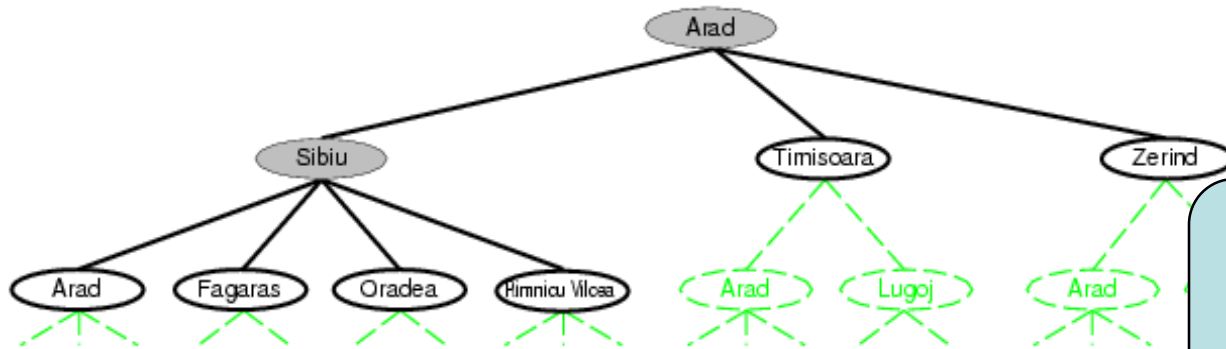
Sub Topics

- Greedy best-first search
- A* search
- Local search algorithms
- Hill-climbing search
- Simulated annealing search
- Local beam search
- Genetic algorithms

Heuristic Search

- Heuristic - a “rule of thumb” used to help guide search
 - often, something learned experientially and recalled when needed
- Heuristic Function - function applied to a state in a search space to indicate a likelihood of success if that state is selected
 - heuristic search methods are known as “weak methods” because of their generality and because they do not apply a great deal of knowledge
 - the methods themselves are not domain or problem specific, only the heuristic function is problem specific
- Heuristic Search –
 - given a search space, a current state and a goal state
 - generate all successor states and evaluate each with our heuristic function
 - select the move that yields the best heuristic value
- Here and in the accompanying notes, we examine various heuristic search algorithms

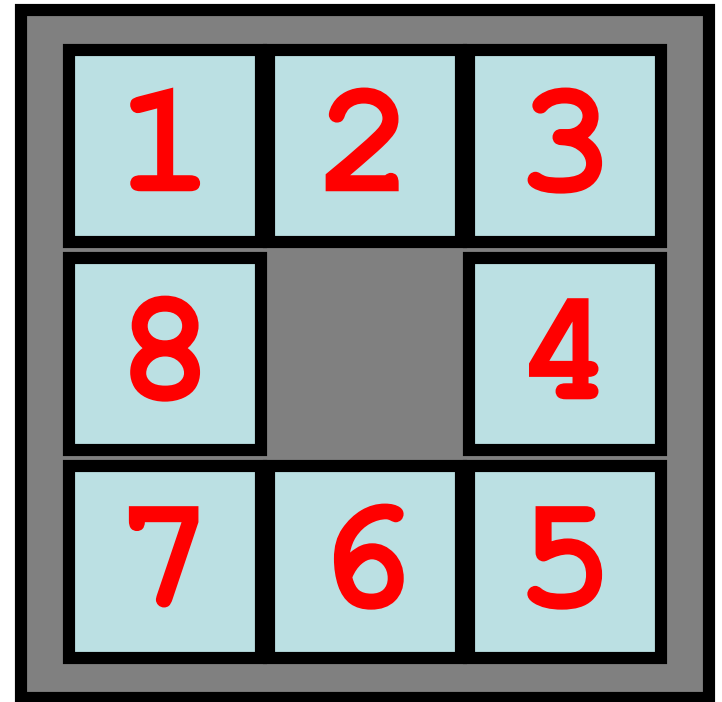
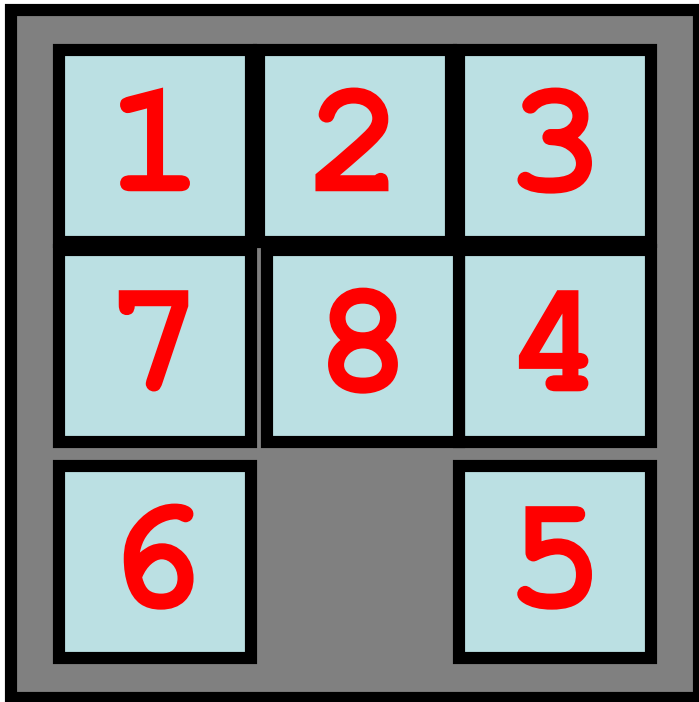
Recall tree search...



This “strategy” is what differentiates different search algorithms

```
function TREE-SEARCH(problem, strategy) returns a solution, failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
```

Example: 8 Puzzle



1	2	3
8		4
7	6	5

GOAL



1	2	3
7	8	4
6		5

left

right

Down

1	2	3
7	8	4
	6	5

1	2	3
7	8	4
6	5	

1	2	3
7		4
6	8	5

Which move is best?

8 Puzzle Heuristics

- Blind search techniques used an arbitrary ordering (priority) of operations.
- Heuristic search techniques make use of domain specific information - a heuristic.
- What heuristic(s) can we use to decide which 8-puzzle move is “best” (worth considering first).

8 Puzzle Heuristics

- For now - we just want to establish some ordering to the possible moves (the values of our heuristic does not matter as long as it ranks the moves).
- Later - we will worry about the actual values returned by the heuristic function.

A Simple 8-puzzle heuristic

- Number of tiles in the correct position.
 - The higher the number the better.
 - Easy to compute (fast and takes little memory).
 - Probably the simplest possible heuristic.

Another approach

- Number of tiles in the *incorrect* position.
 - This can also be considered a lower bound on the number of moves from a solution!
 - The “best” move is the one with the lowest number returned by the heuristic.
 - Is this heuristic more than a heuristic (is it always correct?).
 - Given any 2 states, does it always order them properly with respect to the minimum number of moves away from a solution?

1	2	3
8		4
7	6	5

GOAL



1	2	3
7	8	4
6		5

left

right

Down

1	2	3
7	8	4
	6	5

$h=2$

1	2	3
7	8	4
6	5	

$h=4$

1	2	3
7		4
6	8	5

$h=3$

Another 8-puzzle heuristic

- Count how far away (how many tile movements) each tile is from its correct position.
- Sum up this count over all the tiles.
- This is another estimate on the number of moves away from a solution.

1	2	3
8		4
7	6	5

GOAL



1	2	3
7	8	4
6		5

left

right

up

1	2	3
7	8	4
	6	5

$h=2$

1	2	3
7	8	4
6	5	

$h=4$

1	2	3
7		4
6	8	5

$h=4$

Techniques

- There are a variety of search techniques that rely on the estimate provided by a heuristic function.
- In all cases - the quality (accuracy) of the heuristic is important in real-life application of the technique!

Greedy Best First Search

- always selects the path which appears best at that moment.
- at each step, we can choose the most promising node.

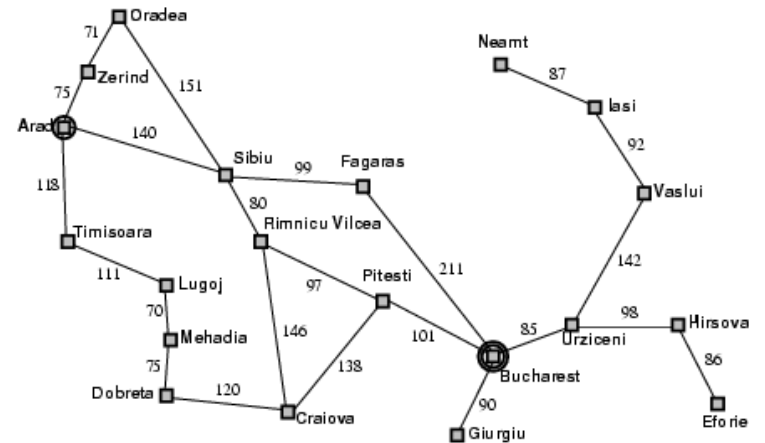
Greedy best-first search

- Evaluation function $f(n) = h(n)$ (**h**euristic)
= estimate of cost from n to *goal*

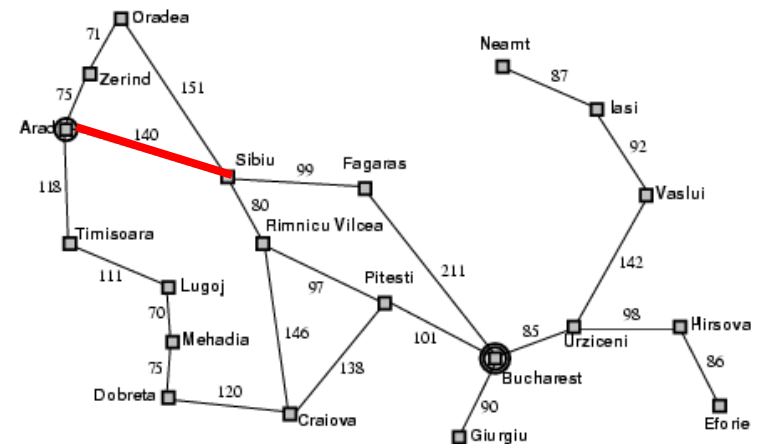
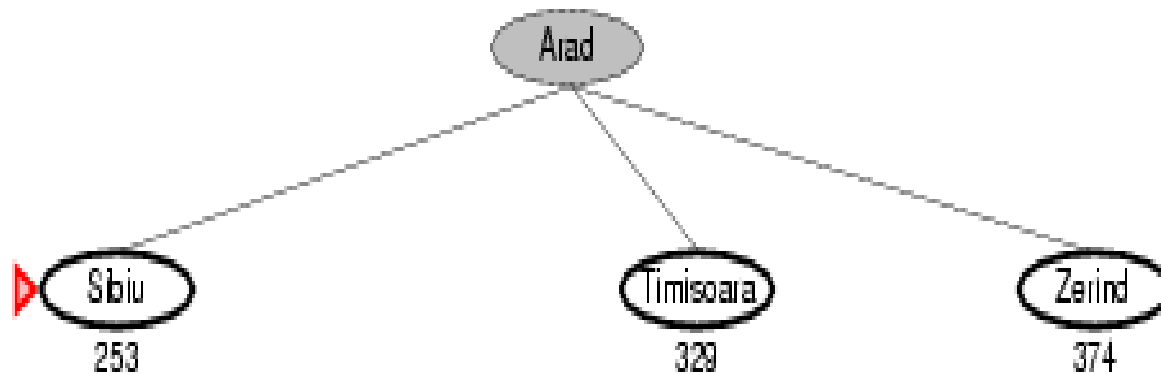
e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest

- Greedy best-first search expands the node that **appears** to be closest to goal

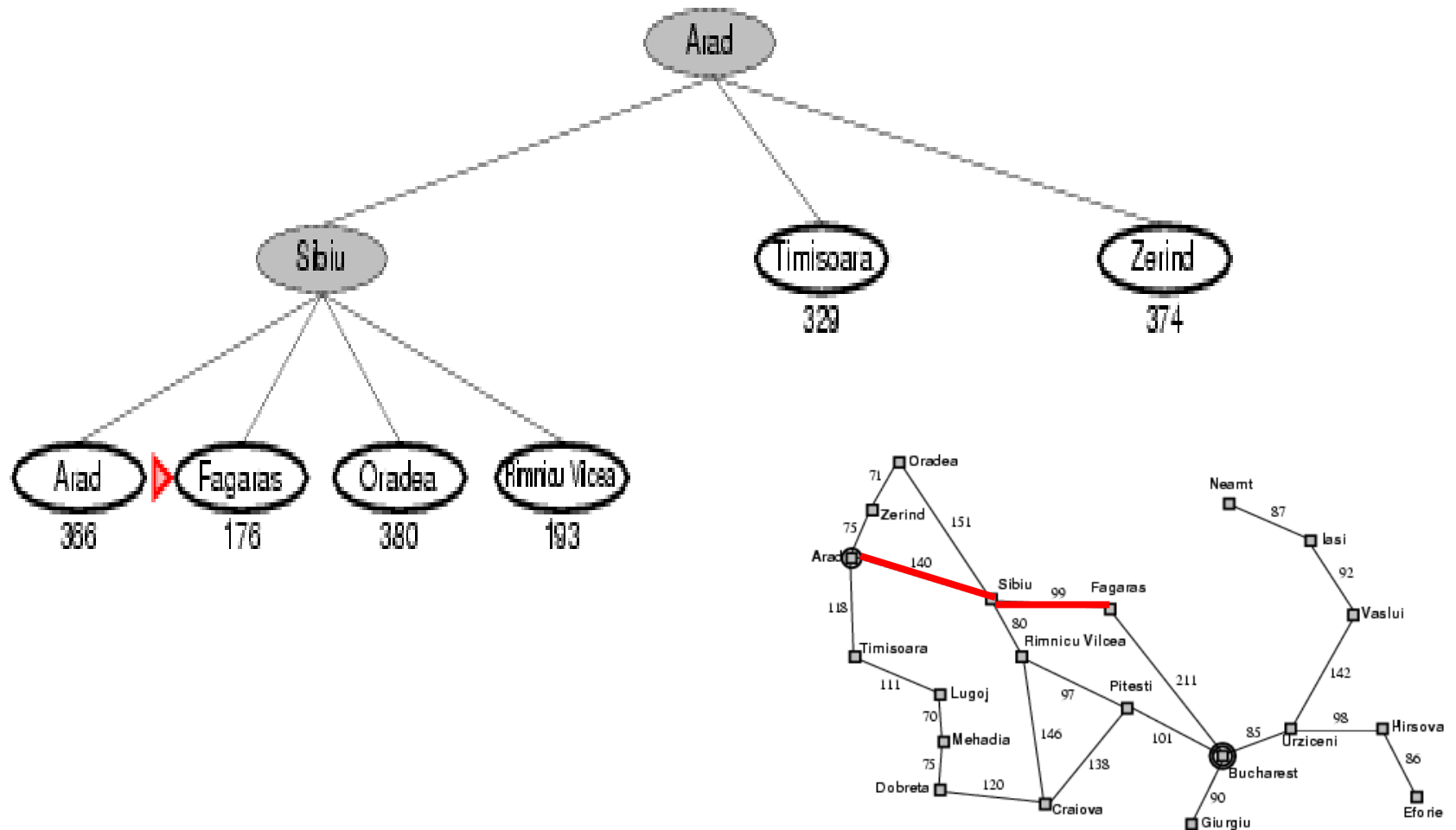
Greedy best-first search example



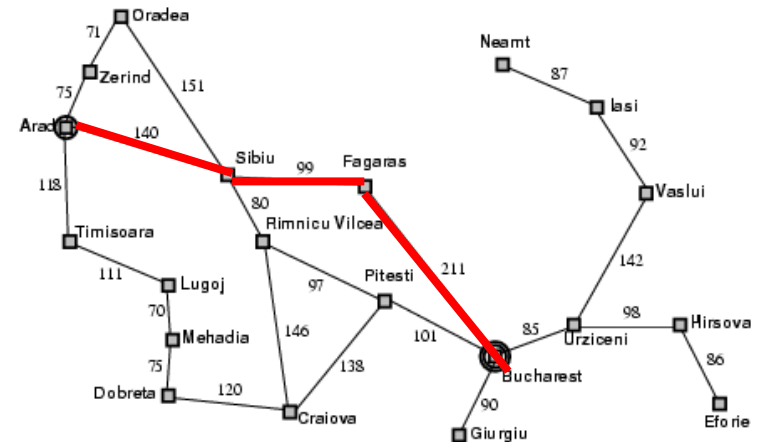
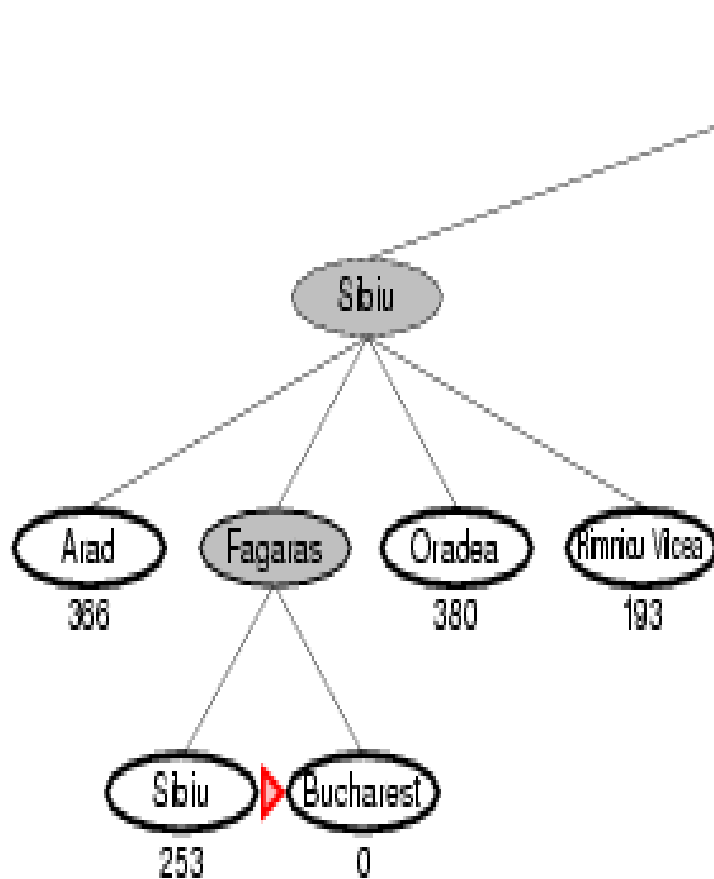
Greedy best-first search example



Greedy best-first search example



Greedy best-first search example



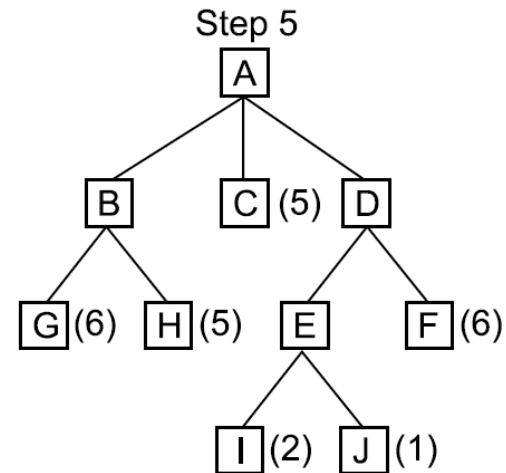
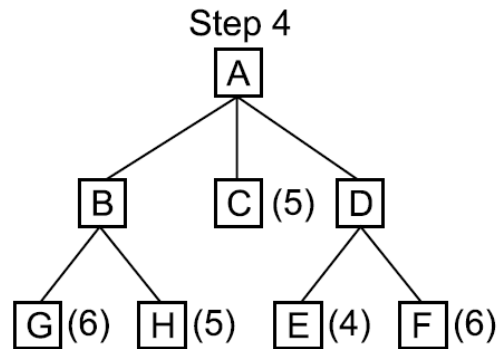
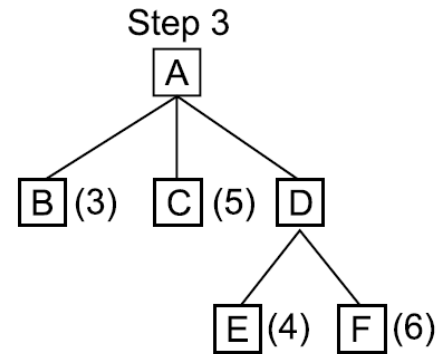
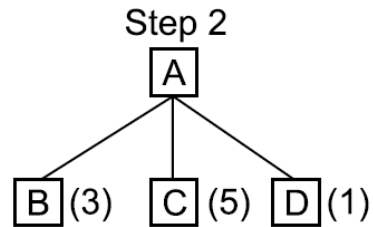
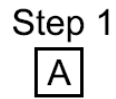
Algorithm (GBFS)

- Open : it is a priority queue maintain to store the node with most promising value of the heuristic function
- Close : node that have already been examined.

GBFS Algorithm

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- **Step 4:** Expand the node n , and generate the successors of node n .
- **Step 5:** Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

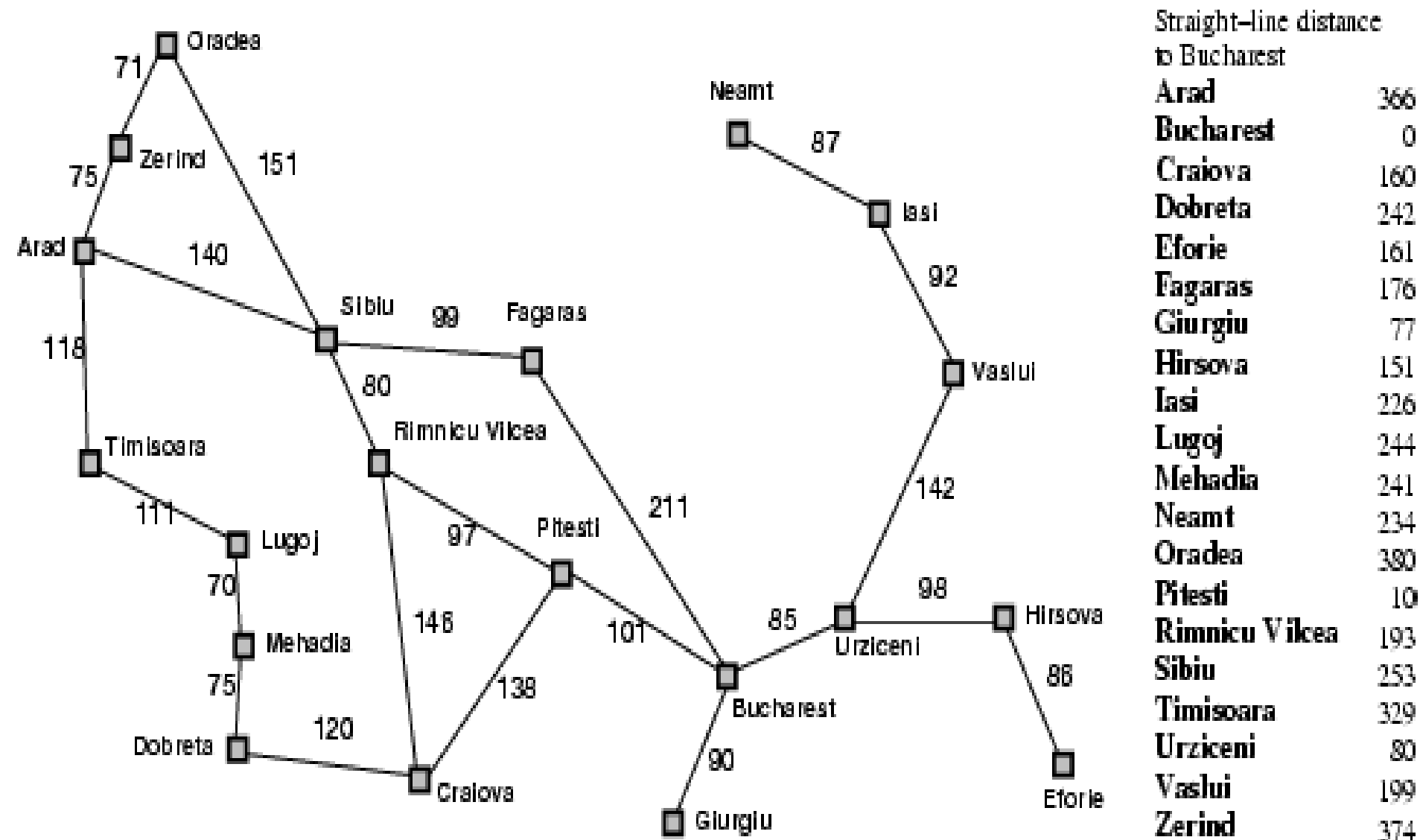
A Best-First Search



Properties of GBFS

- Complete? Not unless it keeps track of all states visited (otherwise can get stuck in loops)
 - e.g., Iasi → Neamt → Iasi → Neamt →
 -
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No e.g. Arad → Sibiu → Rimnicu Virea → Pitesti → Bucharest is shorter!

Romania with step costs in km

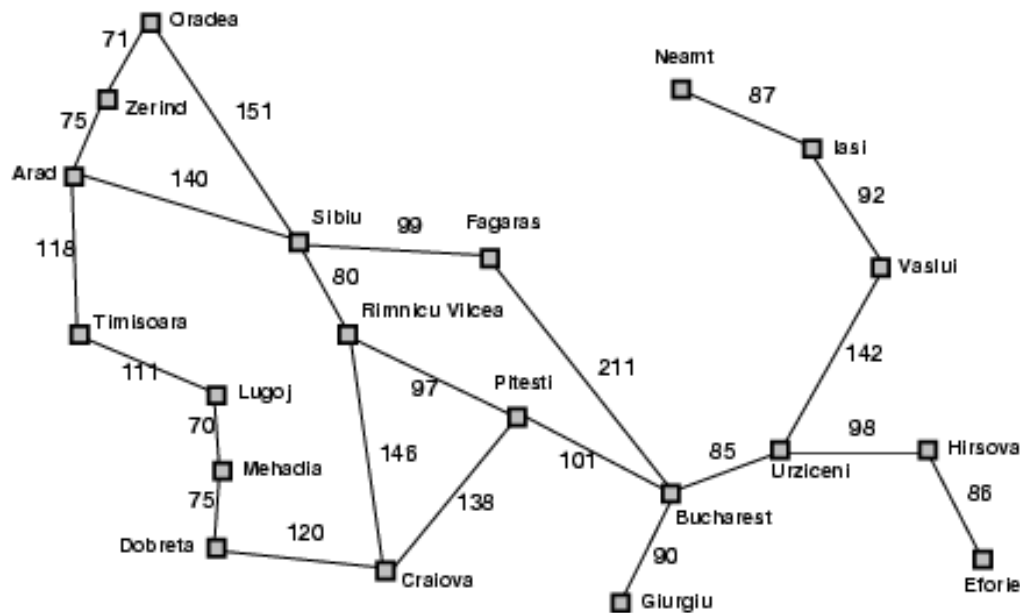


A* search

- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal

A* search example

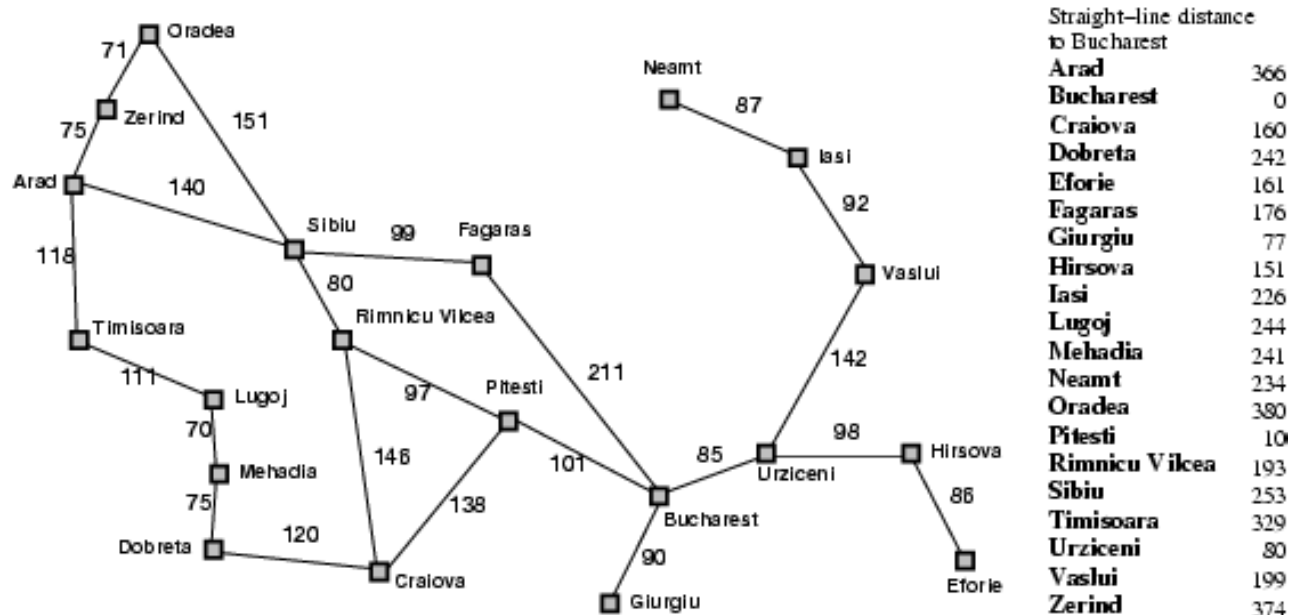
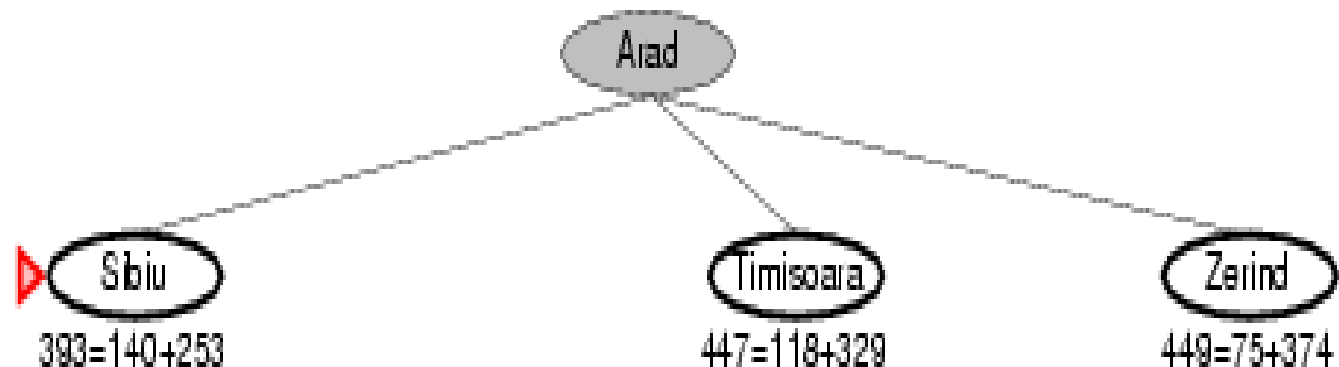
Arad
366=0+366



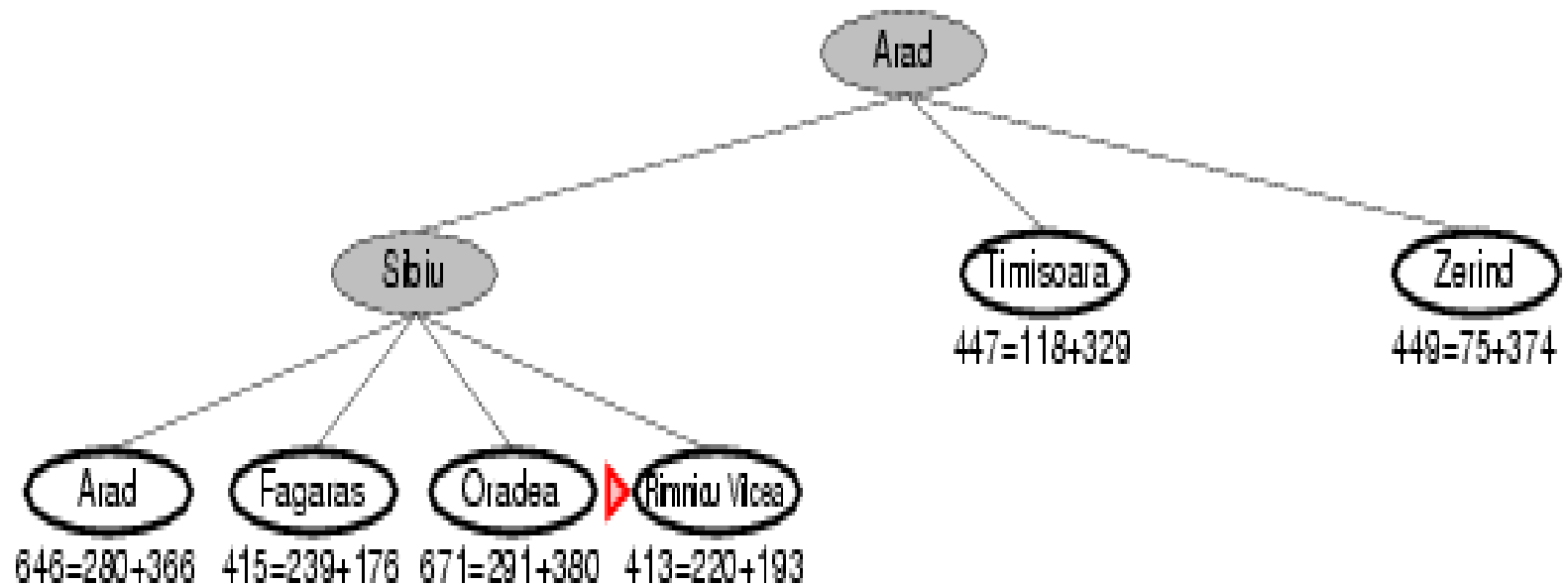
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

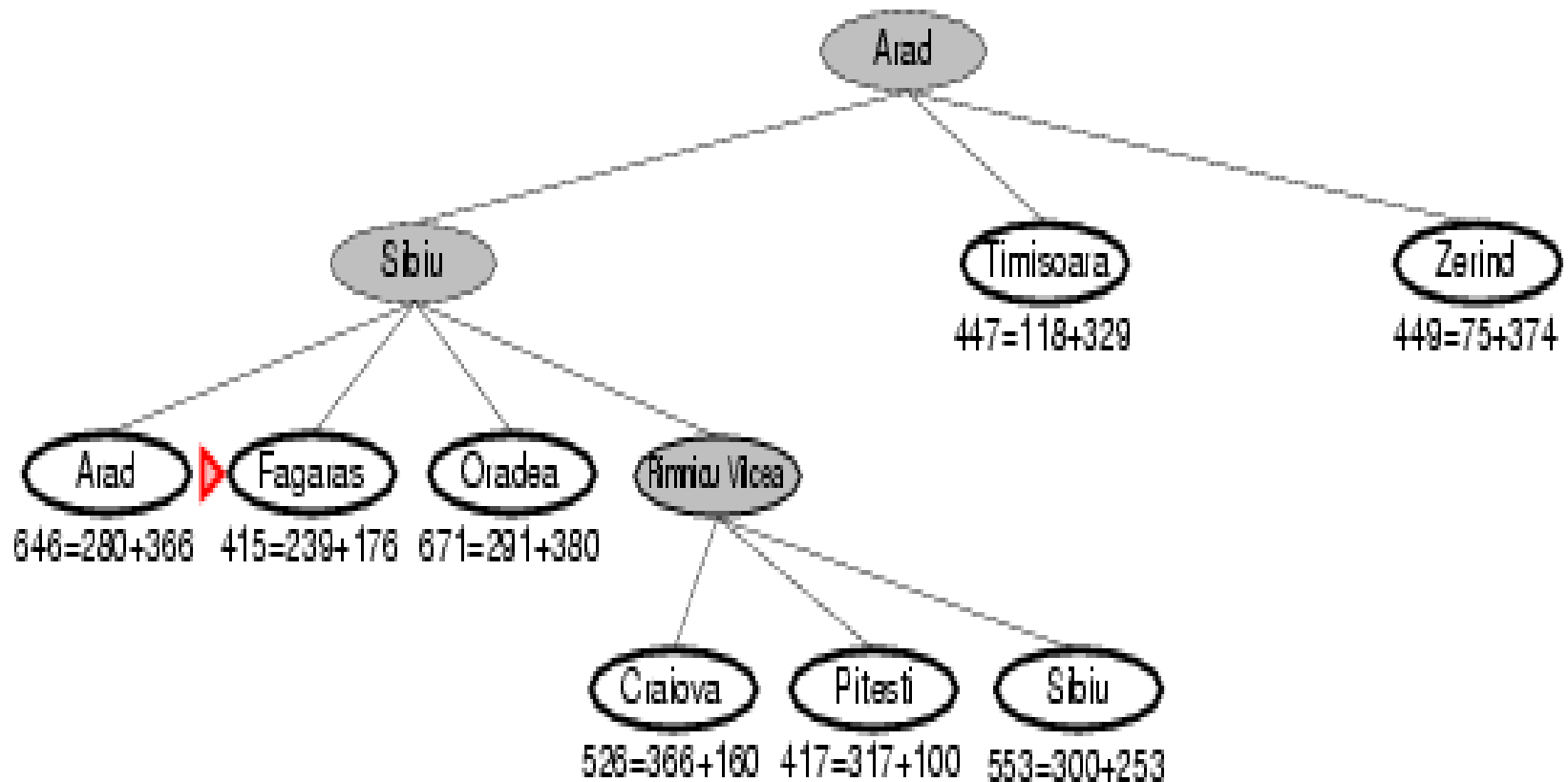
A* search example



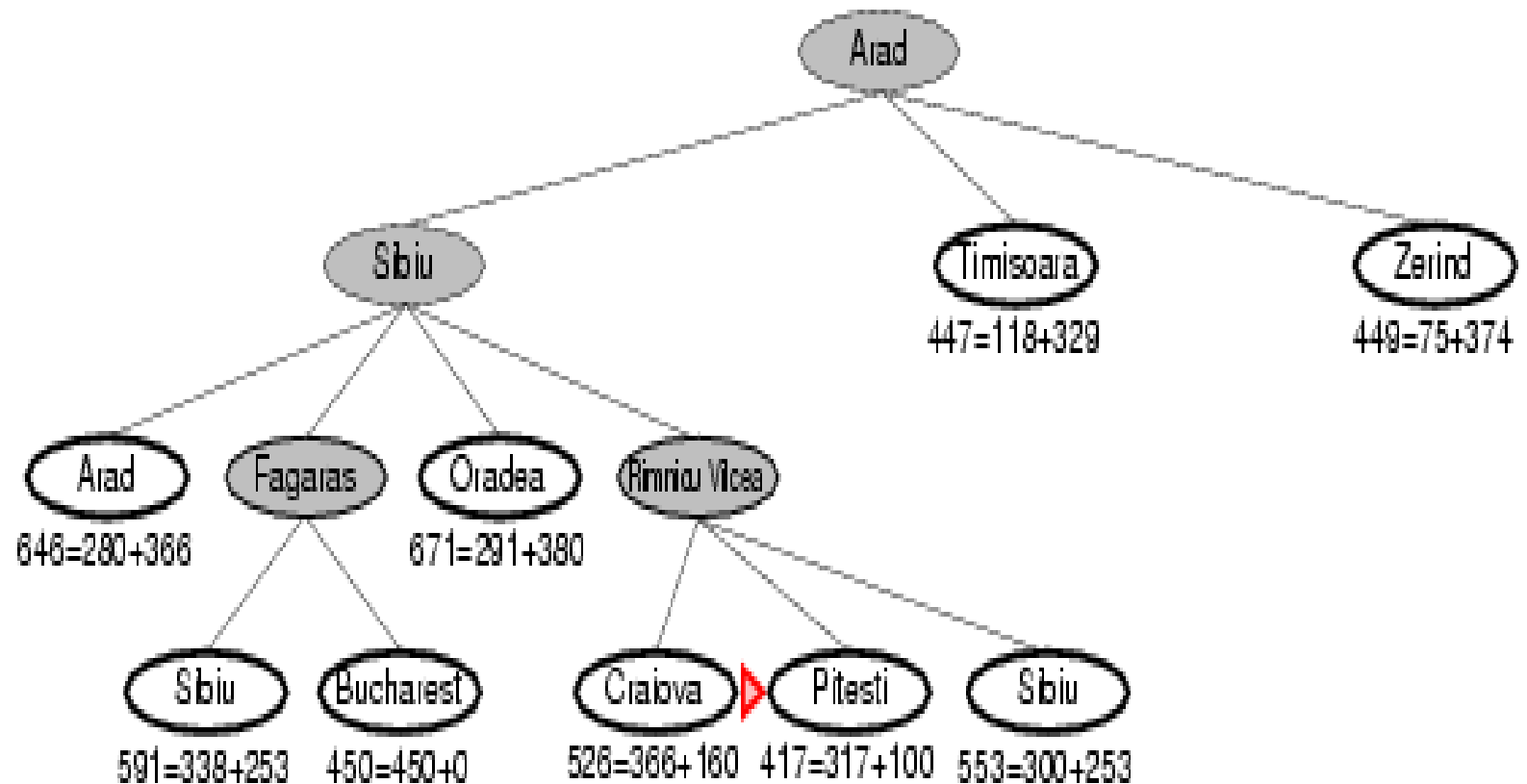
A* search example



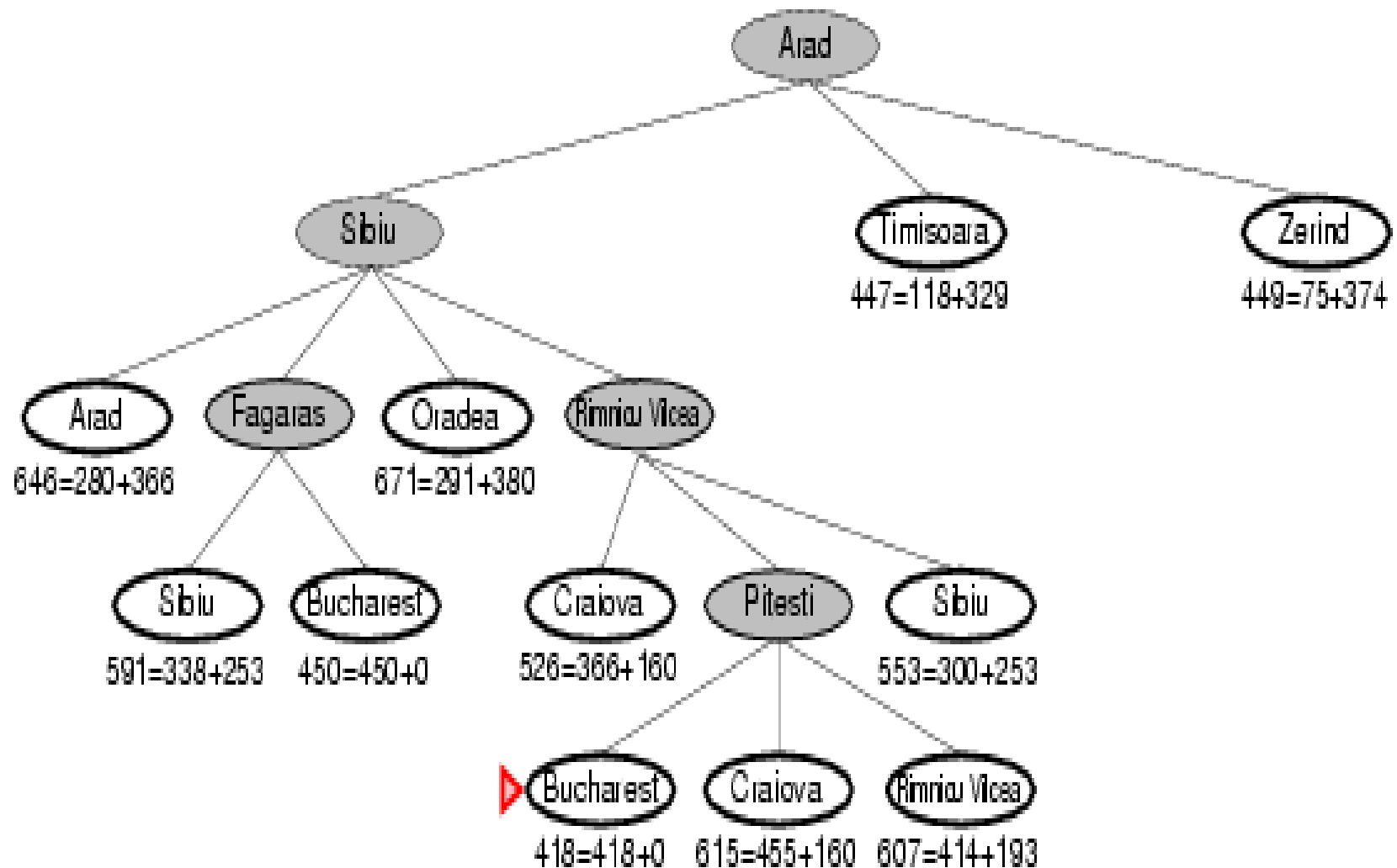
A* search example



A* search example



A* search example



A* Algorithm

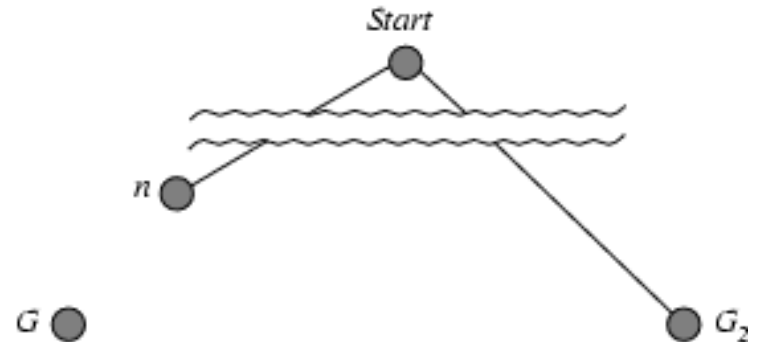
$G(n)$ = cost of n from root, $h(n)$ = cost estimate from n to goal

1. Open={s}, closed={ }, $g(s)=0$, $f(s)=h(s)$
2. If open=empty, return failure
3. Select min cost state n from open, add it to closed
4. If $n \in G$, terminate with success and return $f(n)$
5. For each successor m of n
 1. If m does not belong to (open or closed)// add all successors to open first
 1. Set $g(m)=g(n)+c(n,m)$
 2. Set $f(m)=g(m)+h(m)$
 3. Insert m in open
 2. Else If m belongs to (open or closed)
 1. Set $g(m)=\min(g(m), g(n)+c(n,m))$ // min cost from start to m
 2. Set $f(m)=g(m)+h(m)$
 3. If $f(m)$ has decreased and $m \in$ closed, move m to open
6. Go to step 2

Admissible heuristics

- A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true** cost to reach the goal state from n .
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- **Theorem**: If $h(n)$ is admissible, A^* using TREE-SEARCH is optimal

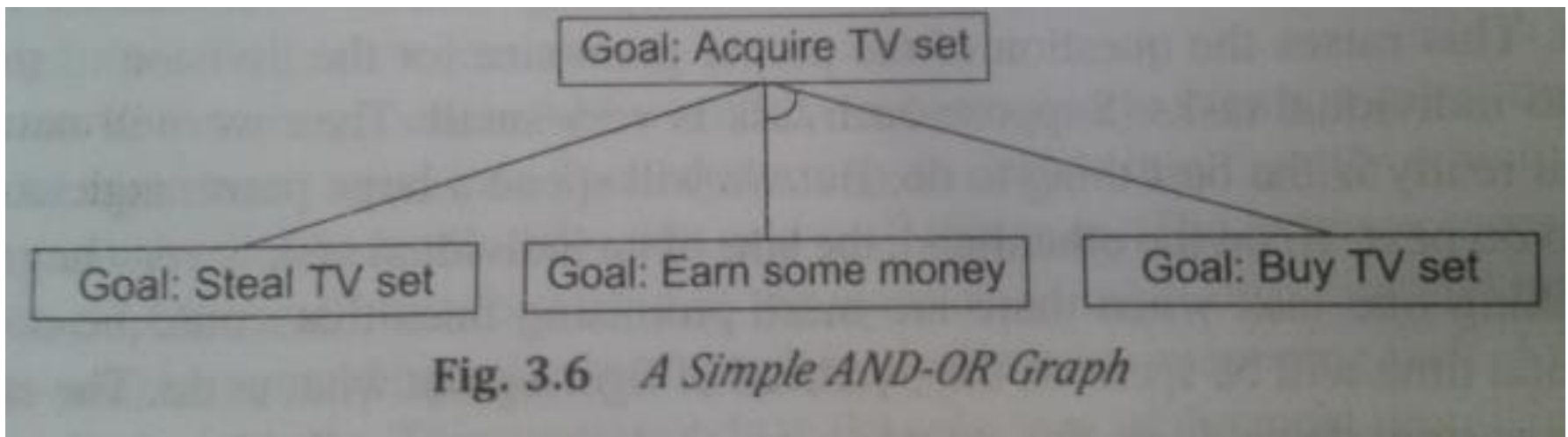
Proof of Admissibility:



- Let G be an optimal goal state
- C^* is the optimal path cost.
- G_2 is a suboptimal goal state: $g(G_2) > C^*$
- Suppose A^* has selected G_2 from OPEN for expansion.
- Consider a node n on OPEN on an optimal path to G .
- Thus $C^* \geq f(n)$
- Since n is not chosen for expansion over G_2 , $f(n) \geq f(G_2)$
- G_2 is a goal state. $f(G_2) = g(G_2)$
- Hence $C^* \geq g(G_2)$.
- This is a contradiction. Thus A^* could not have selected G_2 for expansion before reaching the goal by an optimal path.

Problem Reduction search (And- OR graphs)

- What is problem reduction?
- AND-OR graph (or tree)
 - An OR node represents a choice between possible decomposition
 - An AND node represents a given decomposition



Problem Reduction search (And- OR graphs)

- When a problem can be divided into a set of sub problems, where each sub problem can be solved separately and a combination of these will be a solution, AND-OR graphs or AND - OR trees are used for representing the solution.
- The decomposition of the problem or problem reduction generates AND arcs. One AND arc may point to any number of successor nodes. All these must be solved so that the arc will rise to many arcs, indicating several possible solutions. Hence the graph is known as AND - OR instead of AND. Figure shows an AND - OR graph.

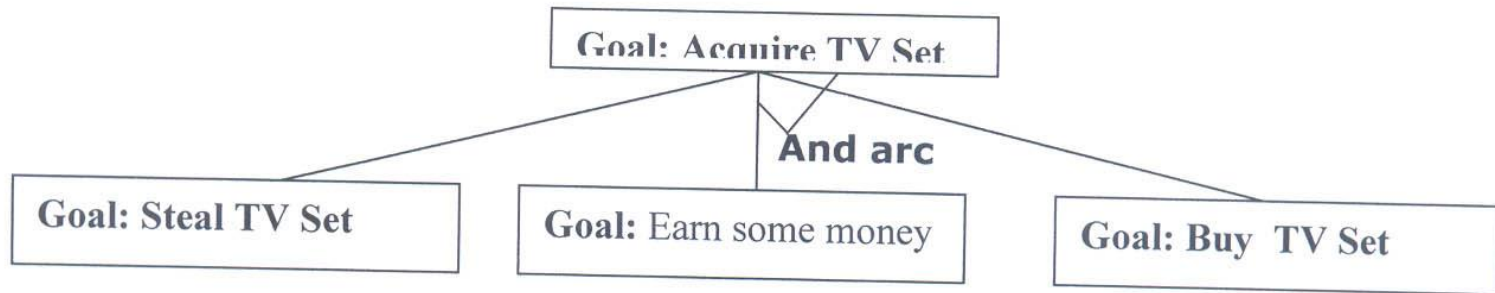


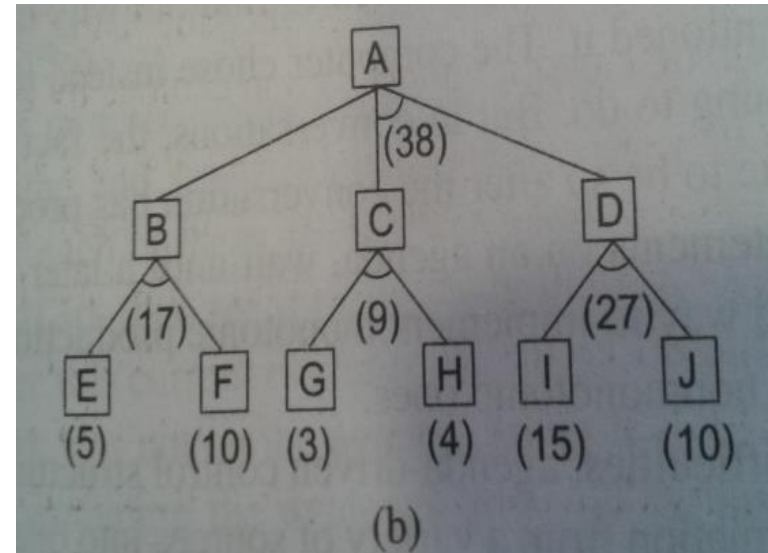
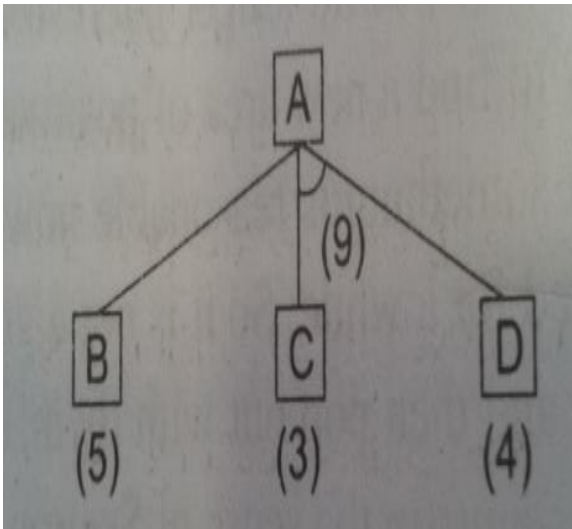
Figure shows AND - Or graph - an example.

Problem Reduction Representation

- Divide and conquer
- Reduce solution recursively to problem to solution to sub-problems
- Problem is solved when all sub-problems are solved
- Example : Matrix Multiplication

Use of GBFS in AND-OR graph

GBFS is not adequate to search in AND-OR Graphs



- In figure (a) the top node A has been expanded producing two area one leading to B and leading to C-D . the numbers at each node represent the value of f' at that node (cost of getting to the goal state from current state). For simplicity, it is assumed that every operation(i.e. applying a rule) has unit cost, i.e., each are with single successor will have a cost of 1 and each of its components.
- With the available information till now , it appears that C is the most promising node to expand since its $f' = 3$, the lowest but going through B would be better since to use C we must also use D' and the cost would be $9(3+4+1+1)$. Through B it would be $6(5+1)$.

Futility

- In order to implement the AND-OR graph algorithm we need a value (threshold)
- FUTILITY : a value that correspond to a threshold such that any solution with cost above is too expensive.
- If the estimated cost of the solution is greater than FUTILITY then abandon the search

Algorithm – Problem Reduction

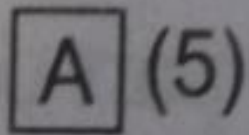
1. Initialize the graph to the starting node
2. Loop until the starting node is labeled SOLVED or until its cost goes above FUTILITY
 1. Traverse the graph,
 1. starting at the initial node
 2. following the current best path,
 3. accumulate the set of nodes that are on that path and have not been expanded or labeled as solved

(Cont....) Problem Reduction

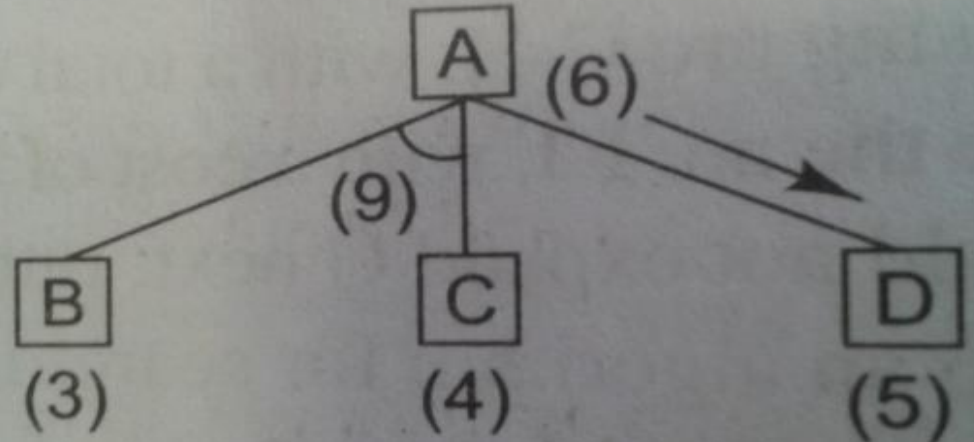
2. Pick one of these unexpanded node and expand it.
 1. If there are no successor, assigned FUTILITY as the value of this node. Otherwise,
 2. add its successor to the graph and for each of them compute f' .
 3. Change the f' estimate of the newly expanded node to reflect the new information provided by successor.
 4. Propagate this change backward through graph.

Operation of Problem Reduction

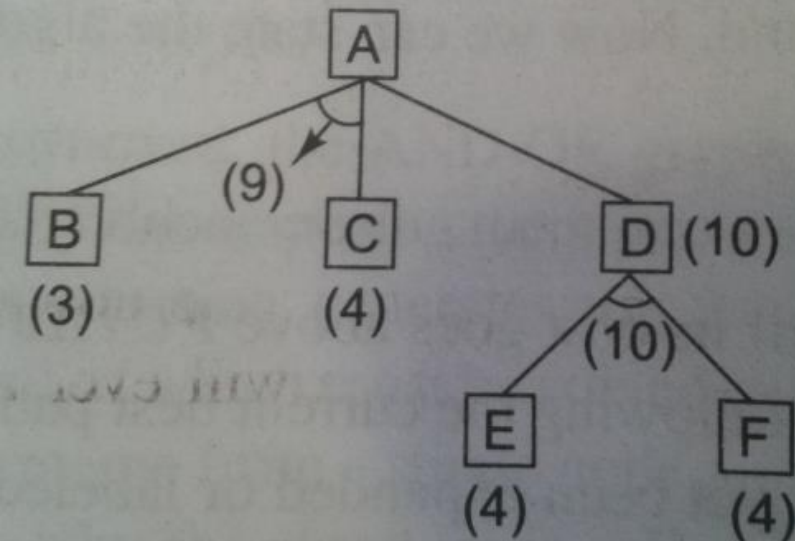
Before step 1



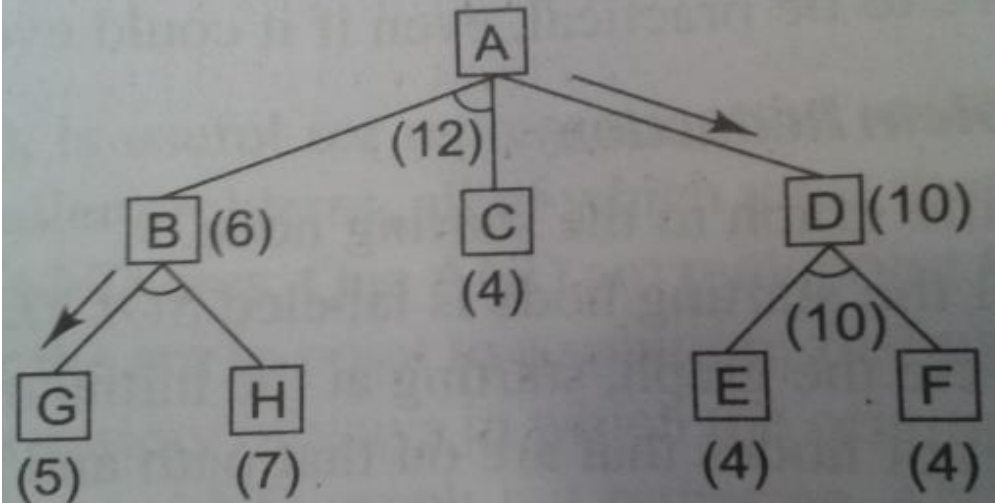
Before step 2



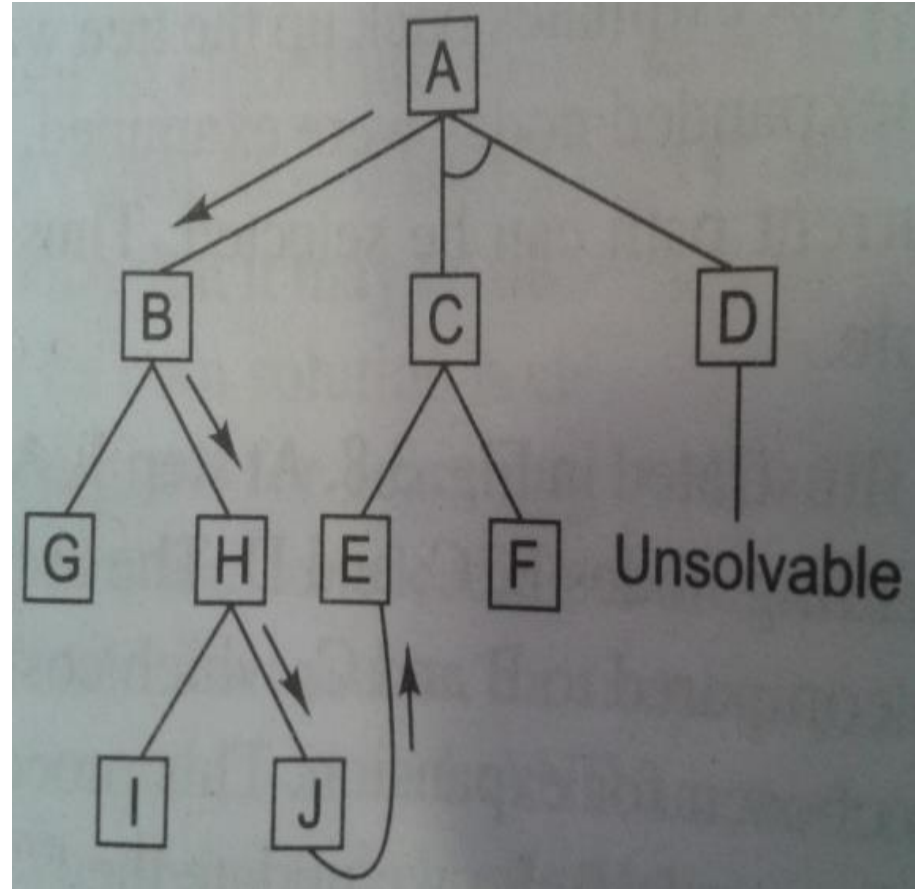
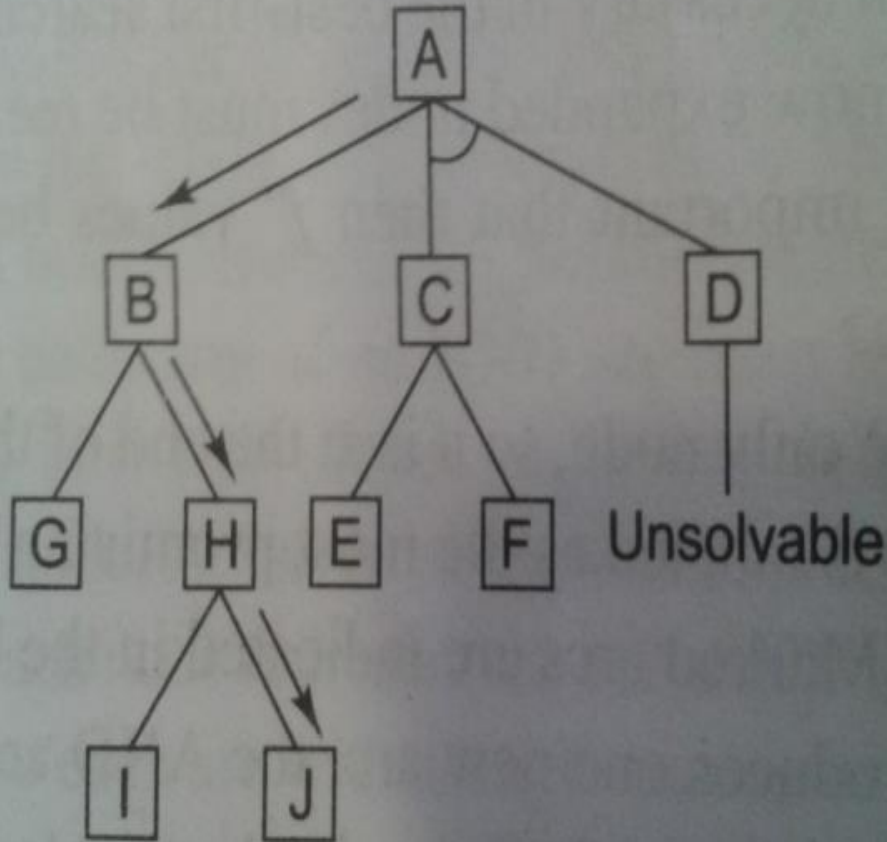
Before step 3



Before step 4



Longer path may be better



The AND-OR graph search problem

- Problem definition
 - Given $[G, s, T]$ where
 - G : Implicitly specified AND/OR graph
 - S : Start node of the AND/OR graph
 - T : Set of terminal nodes
 - $H(n)$ heuristic function estimating the cost of solving the sub-problem at n
 - To find :
 - A minimum cost solution tree

AO* Algorithm

- 1. Let G consists only to the node representing the initial state call this node $INIT$. Compute h' ($INIT$).
- 2. Until $INIT$ is labeled $SOLVED$ or h_i ($INIT$) becomes greater than $FUTILITY$, repeat the following procedure.
 - (I) Trace the marked arcs from $INIT$ and select an unexpanded node $NODE$.
 - (II) Generate the successors of $NODE$. if there are no successors then assign $FUTILITY$ as h' ($NODE$). This means that $NODE$ is not solvable.
 - If there are successors then for each one called $SUCCESSOR$, that is not also an ancestor of $NODE$ do the following
 -

- (a) add SUCCESSOR to graph G
- (b) if successor is a terminal node, mark it solved and assign zero to its h' value.
- (c) If successor is not a terminal node, compute its h' value.
- (III) propagate the newly discovered information up the graph by doing the following . let S be a set of nodes that have been marked SOLVED. Initialize S to NODE. Until S is empty repeat the following procedure;
 - (a) select a node from S call it CURRENT and remove it from S.
 - (b) compute h' of each of the arcs emerging from CURRENT , Assign minimum h' to CURRENT.

- (c) Mark the minimum cost path as the best out of CURRENT.
- (d) Mark CURRENT SOLVED if all of the nodes connected to it through the new marked are have been labeled SOLVED.
- (e) If CURRENT has been marked SOLVED or its h' has just changed, its new status must be propagate backwards up the graph . hence all the ancestors of CURRENT are added to S.

- AO* Search Procedure.
- 1. Place the start node on open.
- 2. Using the search tree, compute the most promising solution tree TP .
- 3. Select node n that is both on open and a part of tp, remove n from open and place it no closed.
- 4. If n is a goal node, label n as solved. If the start node is solved, exit with success where tp is the solution tree, remove all nodes from open with a solved ancestor.
- 5. If n is not solvable node, label n as unsolvable. If the start node is labeled as unsolvable, exit with failure. Remove all nodes from open ,with unsolvable ancestors.
- 6. Otherwise, expand node n generating all of its successor compute the cost of for each newly generated node and place all such nodes on open.
- 7. Go back to step(2)

Algorithm AO*

1. **Initialize** : set $G^* = \{s\}$, $f(s) = h(s)$ (estimate of cost)
 G^* is marked subtree generated till
now
if s belongs T , label s as SOLVED
2. **Terminate** : if s is SOLVED, then Terminate (not decomposable further)
3. **Select** : select a non-terminal leaf node n from the marked sub-tree
4. **Expand** : make explicit the successor of n . For each new successor m , set $f(m) = h(m)$,
if m is terminal, label m as SOLVED
2. **Cost revision** : call Cost-Revision (m)
3. **Loop** : go to step 2

Cost-revision in AO*

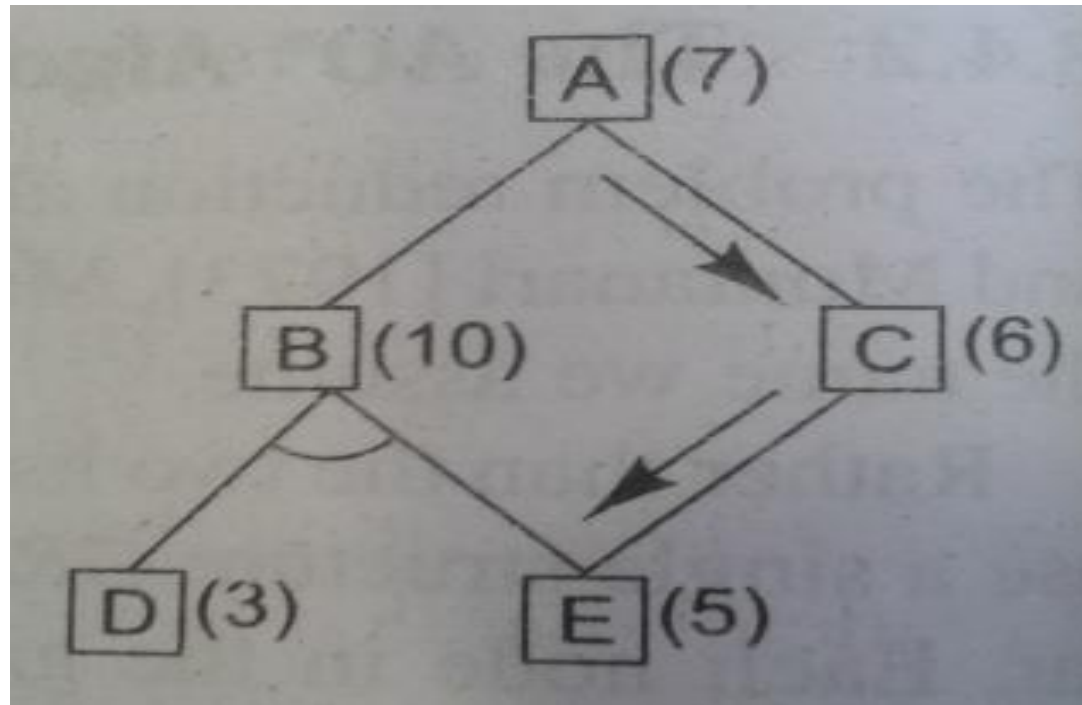
1. Create $z = \{n\}$ // solved nodes
2. If z is empty then return
3. Select a node m from z such that m has no successor in z
4. If m is an AND node with successor $r_1, r_2, \dots, r_n$, set $f(m) = \text{sum of } \{f(r_i) + c(m, r_i)\}$, mark the edge to each successor, if each successor is labeled SOLVED then label m as solved

Contd.....

5. If m is OR node with successor $r_1, r_2 \dots r_k$
 - set $f(m) = \min \{ f(r_i) + c(m, r_i) \}$,
 - mark the edge to the best successor of m ,
 - if marked successor is labeled SOLVED then label m SOLVED
6. If the cost or label of m has changed, then
 - insert those parents of m into z for which m is marked successor

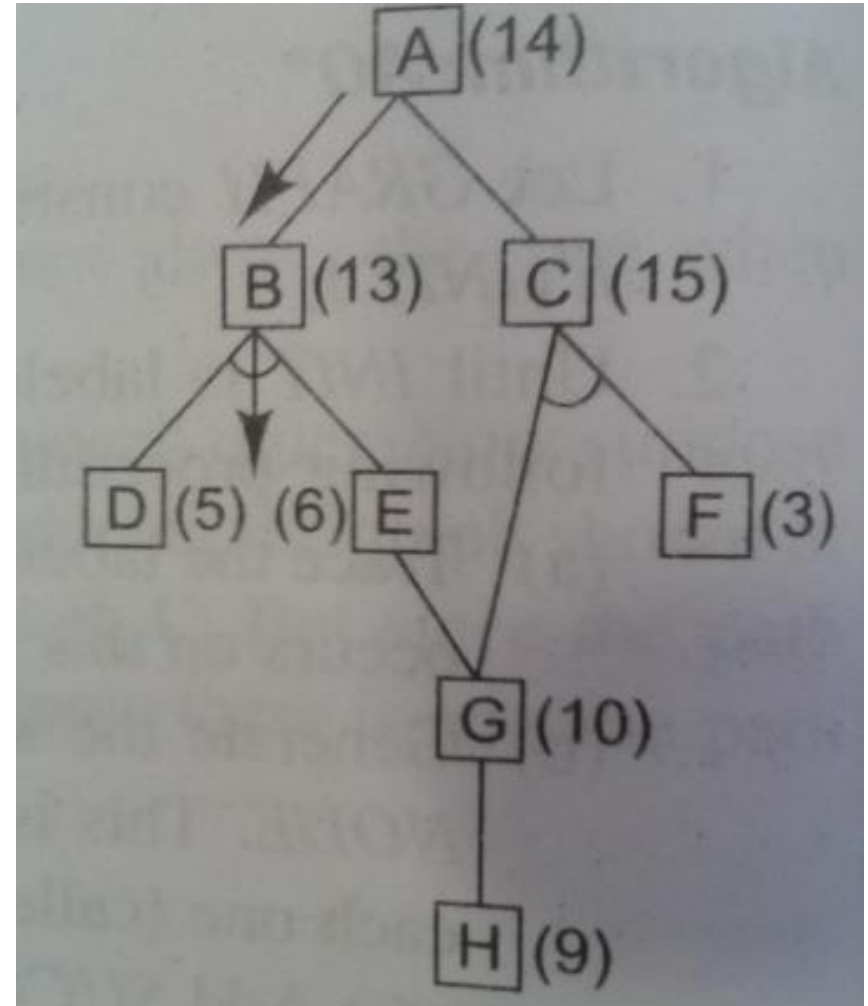
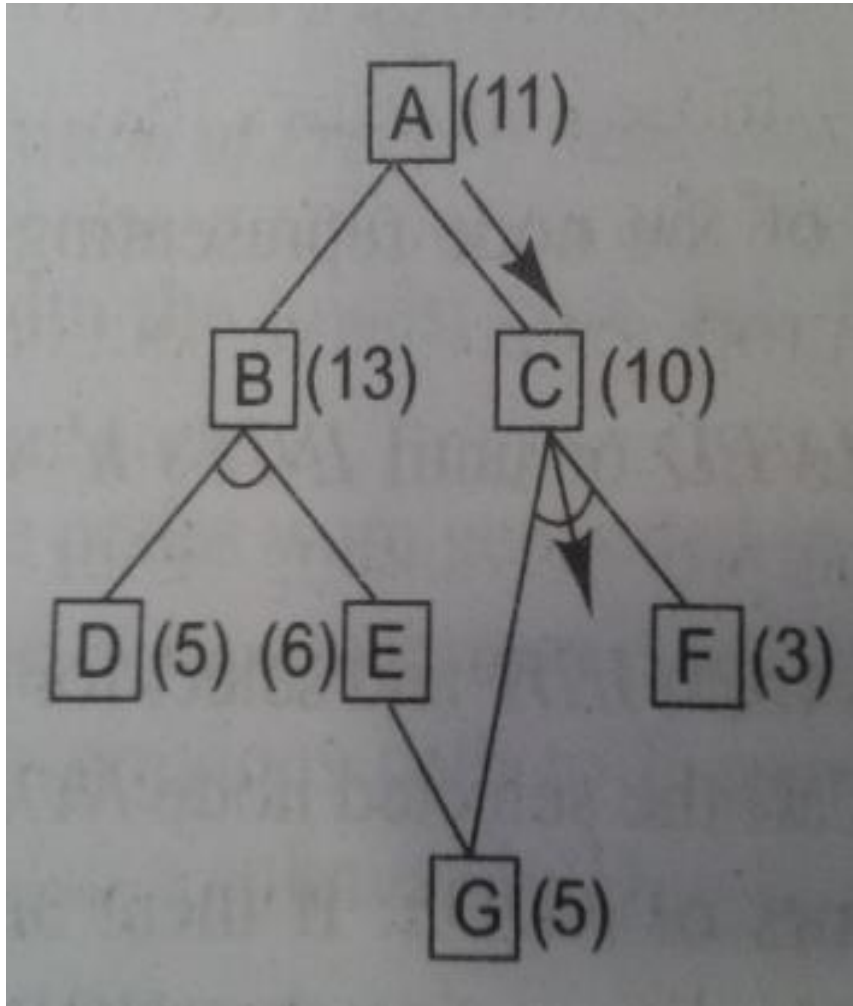
Unnecessary backward propagation

It may revise a cost of the path that is known as not to be very good



Note:- If E is changes to 10 then what?

Necessary backward propagation



Relaxed problems

- A problem with fewer restrictions on the actions is called a **relaxed problem**
-
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
-
- If the rules of the 8-puzzle are relaxed so that a tile can move **anywhere**, then $h_1(n)$ gives the shortest solution
-
- If the rules are relaxed so that a tile can move to **any adjacent square**, then $h_2(n)$ gives the shortest solution

Local search algorithms

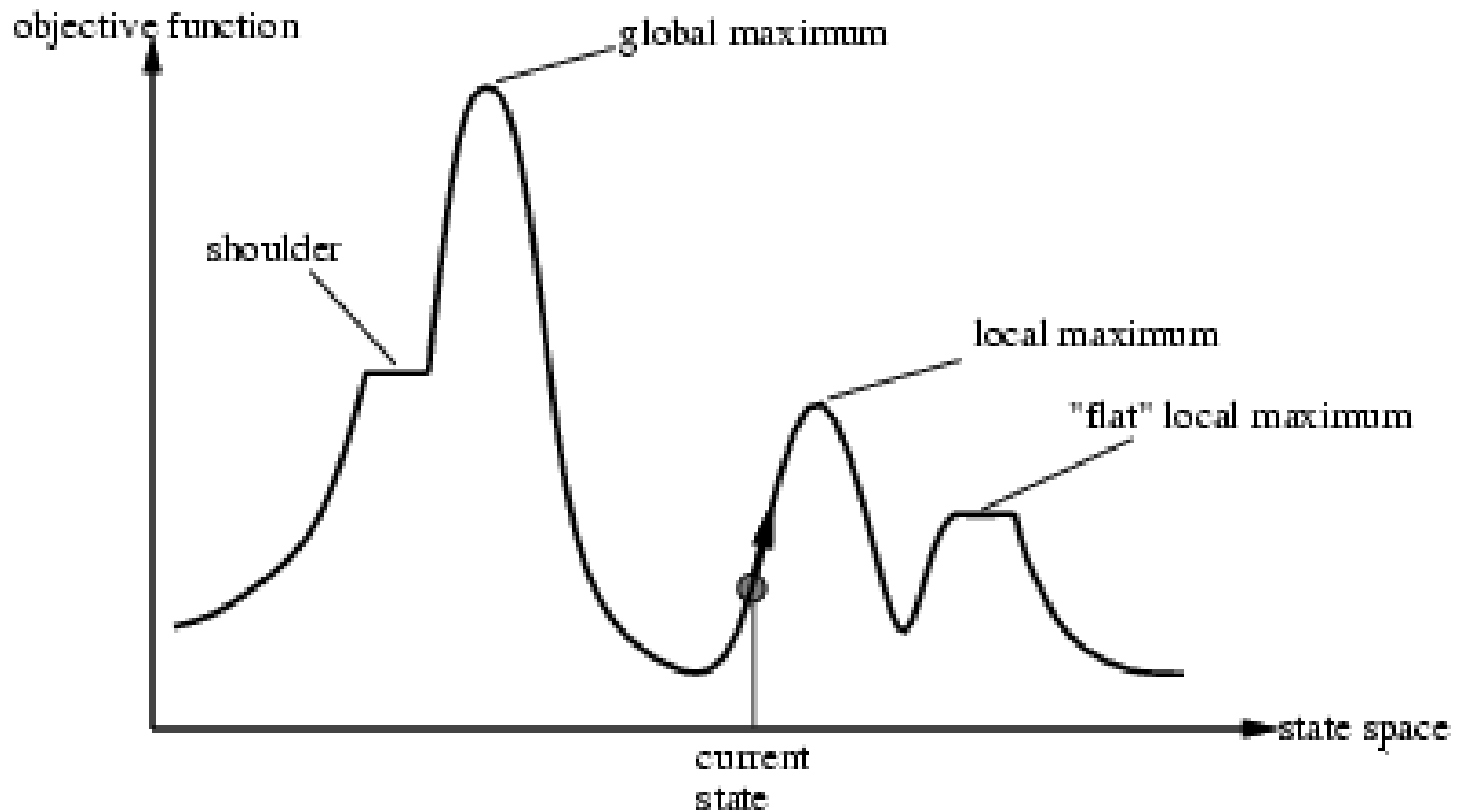
- In many optimization problems, the **path** to the goal is irrelevant; the goal state itself is the solution
- Ex : in 8 queen problem, final configuration is important, not the order in which they are added.
- **Local search** operate using a single current state and generally moves to the neighbors of that state
- Keep a single "current" state, try to improve it
- Local search algorithms are not systematic
- Besides finding the goal, these algorithms are useful for solving pure optimization problems.
- The aim of the optimization problem is to find the best state according to an objective function.

Applications

- VLSI design
- Factory-floor layout
- Job-shop scheduling
- Telecommunication n/w optimization
- Vehicle routing
- Portfolio management

Contd...

- State space Landscape

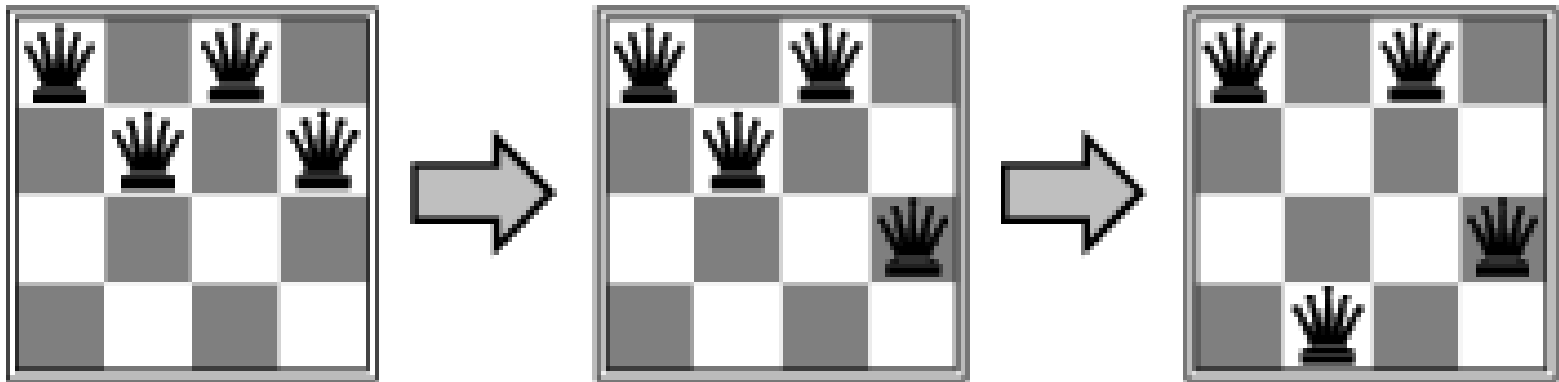


Contd.....

- A landscape has both “location” (define by the state) and elevation (heuristic function or objective function)
- If elevation is correspond to cost them aim is to find lowest valley – Global minimum
- If elevation is correspond to objective function them aim is to find the highest peak – Global maximum
- Local search algorithm explore this landscape

Example: n -queens

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



Key advantages

- They use very little memory-usually a constant amount.
- They can often find reasonable solutions in large or infinite state spaces for which systematic algorithm are unsuitable.
- An optimum algorithm always finds a global maximum/minimum

Hill-climbing search

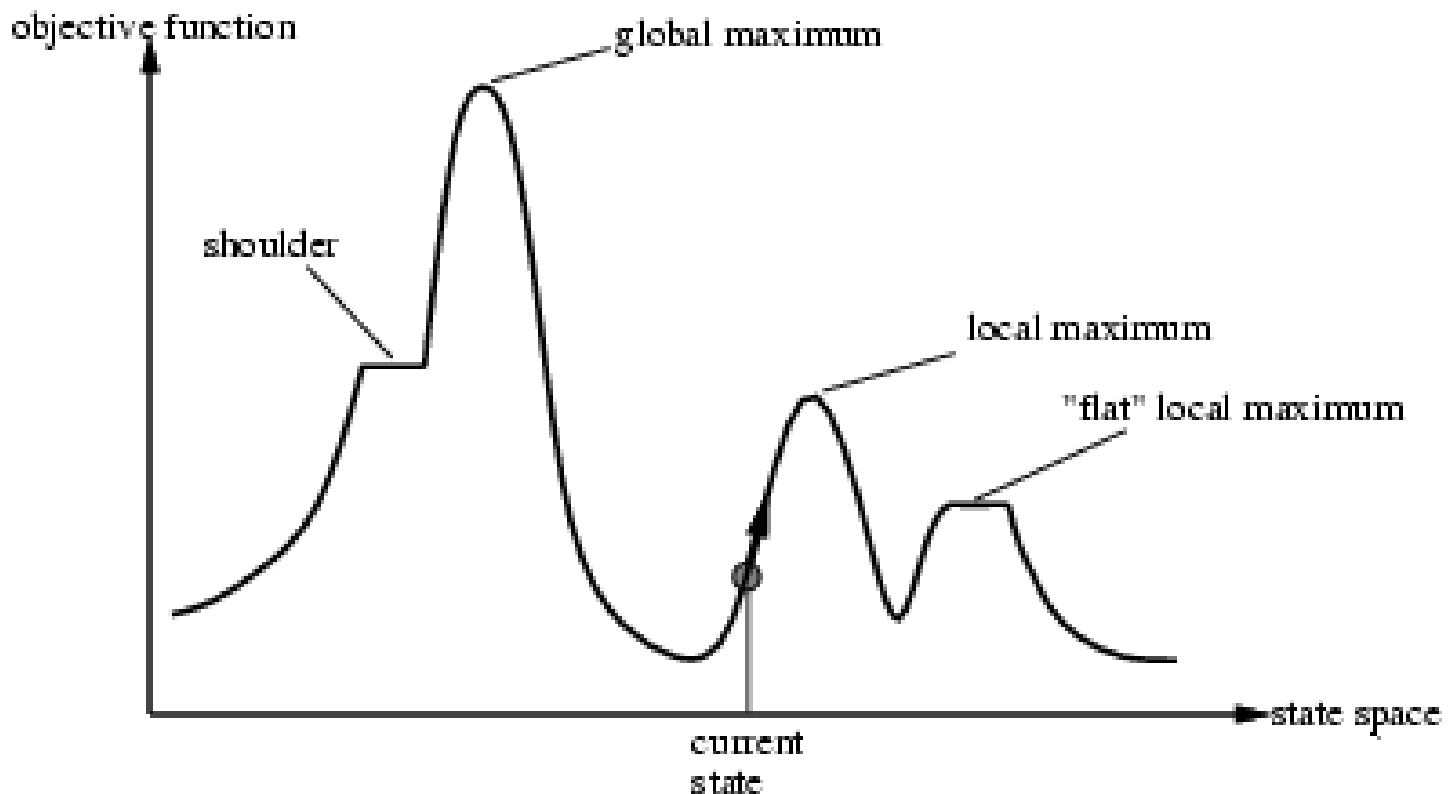
- "Like climbing Everest in thick fog with amnesia"

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```


Hill-climbing search

- Problem: depending on initial state, can get stuck in local maxima



Hill climbing search

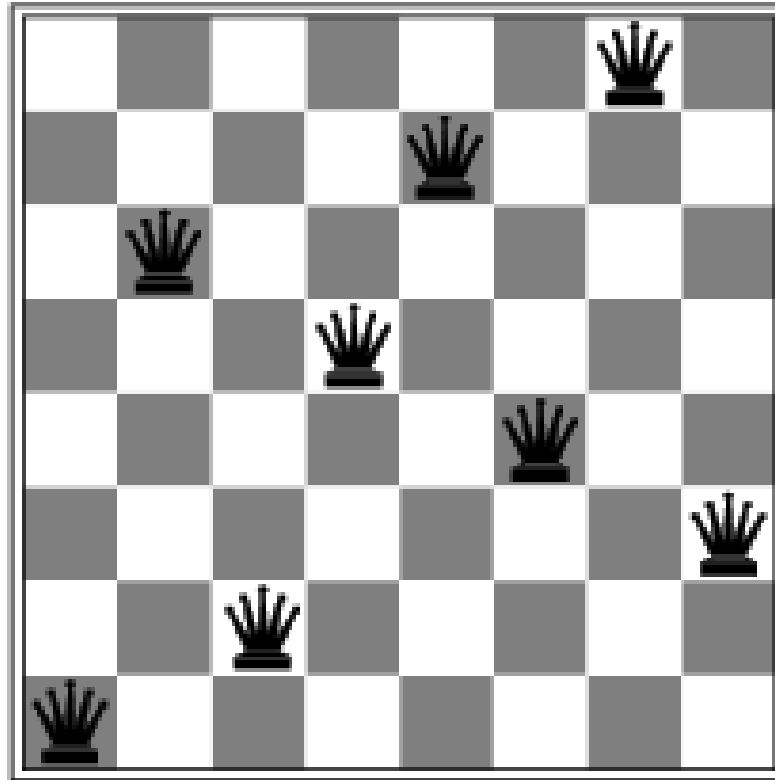
- Greedy local search : it grabs good neighbor state.
- It makes rapid progress towards a solution
- For this algorithm it is easy to improve the bad state.
- In the following example it take 5 steps to reach the state

Hill-climbing search: 8-queens problem

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♚	13	16	13	16
♚	14	17	15	♚	14	16	16
17	♚	16	18	15	♚	15	♚
18	14	♚	15	15	14	♚	16
14	14	13	17	12	14	12	18

- h = number of pairs of queens that are attacking each other, either directly or indirectly
- $h = 17$ for the above state
-

Hill-climbing search: 8-queens problem



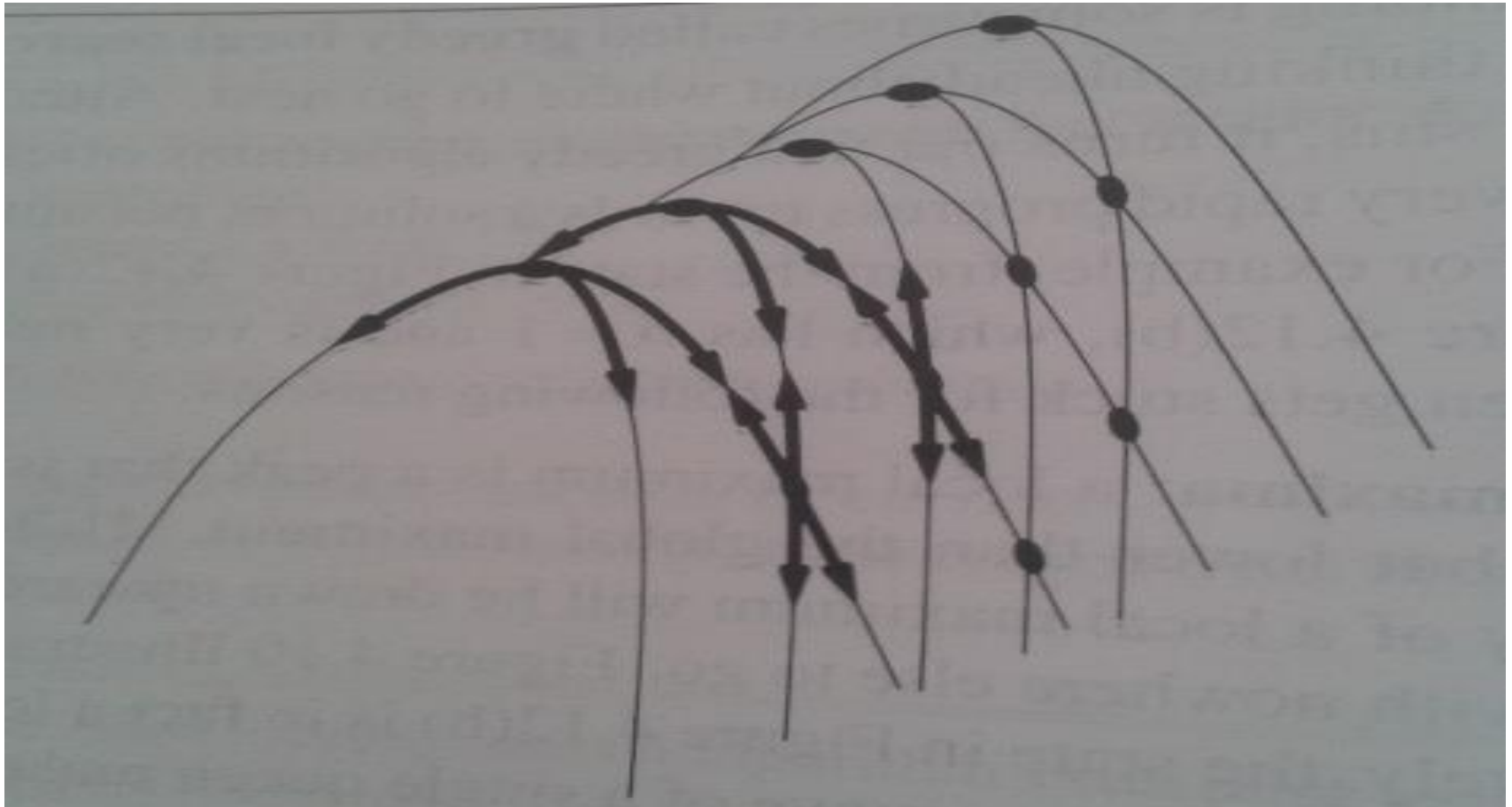
- A local minimum with $h = 1$
- *It near to solution, but it gets stuck.*

Reasons to get stuck in HC

- Local maxima : local maximum is a peak that is higher than each of its neighboring state, but lower than the global maximum
- Ridge : Ridges result in a sequence of local maxima that is very difficult for greedy algorithm to navigate
- Plateau : a plateau is a area of the state space landscape where the evaluation function is flat. It can be flat local maximum, from which no uphill exist, or shoulder from which it is possible to make progress.

Ridge

- Grid of states (dark circles), creating sequence of local maxima. All the available action point downhill



Variants of HC

- Stochastic HC : chooses at random from among uphill moves. Some times it give better solution.
- First-choice HC : implement stochastic hill climbing by generating successor randomly until one is generated that is better than the current state
- Random-restart HC : if at first you don't succeed, try, try again. Conduct series of HC from randomly generated initial states

Random-restart HC

- Stopping when goal is found
- Success of HC depends on shape of the state-space landscape
- Few local maxima and plateaux, random restart HC give good solution very quickly

Simulated annealing search

- Hill climbing search never makes downhill moves
- it guaranteed to be incomplete since can get stuck on a local maximum.
- Purely random walk : moving to the successor chosen uniformly at random from the set of successor : Complete but extremely inefficient.
- Combining the hill climbing with random walk: efficient with completeness - SA search algorithm

Simulated annealing search

- Idea: escape local maxima by allowing some "bad" moves but **gradually decrease** their frequency

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

Properties of simulated annealing search

- One can prove: If T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1
- Widely used in VLSI layout, airline scheduling, etc

Local beam search

- Keeping just one node in the memory is not always a good solution
- Keep track of k states rather than just one
- Start with k randomly generated states
- At each iteration, all the successors of all k states are generated
- If any one is a goal state, stop; else select the k best successors from the complete list and repeat.
- Stochastic beam search (variant)

Local beam search

- It look similar to the random-restart search that it runs in parallel rather in sequence
- In RR each process runs independently of each other
- In LBS useful information is passed among the K parallel search threads
- Ex: one state generate several good state, while other generating all bad moves, first state say to the other – “come over here, the grass is greener”

Local beam search

- The algorithm quickly abandons unfruitful searches
- Move its resources to where more progress is being made
- LBS can suffer from the lack of diversity among the k state
- They can quickly become concentrated in a small region of the state space- making the expensive version of HC

Stochastic Beam Search

- Analogous to stochastic HC
- Instead of choosing best K from the pool of candidate successors
- It selects K successor random, with the probability of choosing a given successor being an increasing function of its value
- It emphasize on natural selection